Feature Extraction Pipeline: How Multi-Frame Video Features Are Extracted

A Technical Explanation for V-JEPA 2 and Qwen2.5-VL

Erize Rai

September 2026

1 Overview

Both V-JEPA 2 and Qwen2.5-VL process **multiple frames per video clip**. However, each model produces a **single feature vector per clip** for the baseline classifiers. The key question is: *how do multiple frames become one vector?*

The answer involves two stages:

1. **Frame Sampling:** Multiple frames are uniformly sampled from each video clip.
2. **Temporal Aggregation:** The model internally processes all frames *jointly* as a spatiotemporal sequence, and the output is pooled (averaged) into a single vector.

2 Stage 1: Frame Sampling (Common to Both Models)

For each video clip, frames are extracted using OpenCV's `VideoCapture`. Frames are sampled **uniformly** across the entire video duration (not consecutive):

```
def load_frames_from_file(video_path, n_frames=8, size=224):
    cap = cv2.VideoCapture(video_path)
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    idxs = np.linspace(0, total - 1, n_frames, dtype=int)
    frames = []
    for i in idxs:
        cap.set(cv2.CAP_PROP_POS_FRAMES, i)
        ok, fr = cap.read()
        fr = cv2.cvtColor(fr, cv2.COLOR_BGR2RGB)
        fr = cv2.resize(fr, (size, size))
        frames.append(Image.fromarray(fr))
```

```
    return frames
```

Key detail: `np.linspace(0, total-1, n_frames)` produces **evenly-spaced** frame indices. For example, a 120-frame video sampled at $n = 8$ would select frames at indices [0, 17, 34, 51, 68, 85, 102, 119].

- **V-JEPA 2:** 8 frames, resized to 224×224
- **Qwen2.5-VL:** 4 frames, resized to 224×224

3 Stage 2: V-JEPA 2 Feature Extraction

3.1 How the Model Processes Multiple Frames

V-JEPA 2 does **not** process each frame independently. The 8 frames are treated as a **3D video volume** and tokenized jointly:

1. The 8 frames ($8 \times 224 \times 224 \times 3$) are passed to the `VJEPA2VideoProcessor`, which converts them into a single 4D pixel tensor.
2. A **3D convolutional patch embedding** (`Conv3D`) with tubelet size $2 \times 16 \times 16$ converts the video into spatiotemporal tokens:
   - Temporal dimension: $8 \div 2 = 4$ temporal tokens ($T' = 4$)
   - Spatial dimensions: $224 \div 16 = 14$ per axis, so $14 \times 14 = 196$ spatial tokens
   - Total tokens: $4 \times 196 = 784$ tokens, each of dimension 1280
3. These 784 tokens pass through **all 32 transformer layers** of ViT-Huge as a single sequence — the self-attention operates over *all* spatiotemporal positions jointly, meaning **every frame attends to every other frame**.

3.2 How 784 Tokens Become One Vector (Baseline Classifiers)

For the baseline classifiers (SVM, RF, LR, MLP, etc.), a single 1280-d vector per clip is needed. This is achieved via **global mean pooling over all 784 tokens** from the last hidden layer:

```
# out.last_hidden_state shape: (1, 784, 1280)
single_feat = out.last_hidden_state.mean(dim=1)  # -> (1, 1280)
```

This averages across **all** 784 spatiotemporal tokens (all 4 temporal positions $\times$ all 196 spatial positions), producing a single 1280-dimensional feature vector that represents the entire 8-frame clip.

3.3 How 784 Tokens Become Multi-Layer Features (HAF-STD Net)

For the HAF-STD Net, the extraction preserves temporal structure:

1. Hidden states are extracted from **4 intermediate layers** at 25%, 50%, 75%, and 100% depth (layers 8, 16, 24, 32 for ViT-Huge).
2. At each layer, the 784 tokens are **reshaped** into a 4×196 grid (temporal $\times$ spatial).
3. **Spatial mean pooling** averages across the 196 spatial positions per time step, yielding a $(T' = 4, D = 1280)$ matrix per layer.
4. Stacking across 4 layers: final shape is $(4_{\text{layers}}, 4_{\text{frames}}, 1280)$ per clip.

```
# For each selected layer:
h = all_hidden[li].float()                # (1, 784, 1280)
h = h.view(1, 4, 196, 1280)               # reshape: (1, T', S, D)
h_temporal = h.mean(dim=2)                # spatial pool: (1, 4,
    1280)
per_layer_frames.append(h_temporal)

# Stack all 4 layers:
stacked = torch.stack(per_layer_frames) # (4, 4, 1280)
```

4 Stage 2: Qwen2.5-VL Feature Extraction

4.1 How the Model Processes Multiple Frames

Qwen2.5-VL takes **4 frames** per clip and processes them via a vision-language pipeline:

1. The 4 frames are formatted as a chat message with a minimal prompt ("Describe.").
2. The **Vision Encoder** converts each frame into visual tokens via 14×14 patches with a Conv3D temporal grouping (stride 2), producing visual tokens.
3. These visual tokens are projected into the LM embedding space (3584-d) and **interleaved with text tokens** (the prompt).
4. The full multimodal sequence passes through the **Qwen2.5 LM Decoder** (28 transformer layers).

4.2 How the Token Sequence Becomes One Vector

The hidden states from the **final decoder layer** are mean-pooled across the *entire* sequence dimension (both visual and text tokens):

```
# out.hidden_states[-1] shape: (1, seq_len, 3584)
feat = out.hidden_states[-1].mean(dim=1)  # -> (1, 3584)
```

This produces a single **3584-dimensional** feature vector per clip.

5 Summary: The Complete Pipeline

Stage	V-JEPA 2	Qwen2.5-VL
Input frames	8 frames at 224×224	4 frames at 224×224
Tokenization	Conv3D tubelets ($2 \times 16 \times 16$) $\rightarrow$ 784 tokens	Conv3D patches (14×14, stride 2) $\rightarrow$ variable tokens
Encoder	32-layer ViT-Huge (654M params)	Vision Encoder + 28-layer LM Decoder (7B params, 4-bit)
Temporal handling	All 8 frames jointly via 3D spatiotemporal attention	Frames interleaved with text in autoregressive decoder
Baseline feature	Mean pool over all 784 tokens $\rightarrow$ **1280-d**	Mean pool over all sequence tokens $\rightarrow$ **3584-d**
HAF-STD feature	4 layers $\times$ spatial pool $\rightarrow$ **(4, 4, 1280)**	Not used (V-JEPA 2 only)

6 Why Only 8 Frames from Videos with Hundreds of Frames?

A natural question arises: if a video clip has 100–1000+ frames (at 24–30 fps), why sample only 8? There are four complementary reasons:

6.1 The Model Architecture Requires It

V-JEPA 2 was **pretrained** with 8-frame input clips. Its Conv3D tubelet embedding layer ($2 \times 16 \times 16$) is architecturally designed for this temporal depth. The tubelet's temporal stride of

2 over 8 frames produces exactly $T' = 4$ temporal tokens — this is a fixed architectural parameter, not an arbitrary choice. Using a different number of frames would produce a mismatched token sequence that does not align with the pretrained positional embeddings and attention patterns.

Similarly, Qwen2.5-VL's video processing pipeline uses 4 frames by default, matching its Conv3D temporal grouping stride.

6.2 Uniform Sampling Covers the Full Temporal Span

The 8 frames are **not consecutive** — they are **uniformly spaced** across the entire video duration:

```
idxs = np.linspace(0, total - 1, n_frames, dtype=int)
```

For a 240-frame video (8 seconds at 30 fps), the 8 selected frame indices would be:

$$[0,\ 34,\ 68,\ 103,\ 137,\ 171,\ 205,\ 239]$$

This means the model sees the **beginning, middle, and end** of the entire clip. Each sampled frame is separated by approximately 1 second, covering the full temporal extent of the action.



8 uniformly-sampled frames (black dots) from a 240-frame video timeline.

6.3 GPU Memory Constraints

Self-attention has **quadratic complexity** $\mathcal{O}(N^2)$ in the number of tokens. With 8 frames, V-JEPA 2 produces 784 tokens. Doubling to 16 frames would produce 1,568 tokens, requiring approximately $4\times$ more memory for attention computation — infeasible on the 15.6 GB Tesla T4 GPU used in this study, especially alongside the 4-bit quantized 7B-parameter Qwen2.5-VL model.

Frames	Tokens	Relative Attention Cost
4	392	$0.25\times$
8	784	$1.0\times$ (used)
16	1568	$4.0\times$
32	3136	$16.0\times$

6.4 Temporal Redundancy in Video

At 24–30 fps, adjacent frames differ by only 33–42 milliseconds — the visual difference between consecutive frames is **negligible**. For action recognition tasks like distinguishing Balance Beam from Vault, the discriminative information lies in *key body poses and transitions*, not in every sub-frame motion. Eight uniformly-spaced frames capture the essential "stroboscopic" motion trajectory.

6.5 This Is the Field-Wide Standard

Virtually all modern video transformers use 8–16 sampled frames:

Model	**Default Frames**
V-JEPA 2 / V-JEPA 2.1 (Meta)	8
VideoMAE (Tong et al., 2022)	16
TimeSformer (Bertasius et al., 2021)	8
ViViT (Arnab et al., 2021)	8–32
Qwen2.5-VL (Alibaba)	4

This is not a limitation of the present study — it is the established methodology for frozen-feature video representation evaluation.

7 Key Takeaway

Multiple frames are NOT processed independently. Both models treat multi-frame input as a *joint spatiotemporal sequence*. The self-attention in V-JEPA 2's ViT-Huge operates over all 784 tokens from all 8 frames simultaneously — frame 1's patches attend to frame 8's patches. The final feature vector is obtained by **mean-pooling** this joint representation, which inherently captures cross-frame temporal relationships learned during pretraining. This is fundamentally different from extracting a per-frame feature and averaging afterward.

The choice of 8 frames is architecturally determined by the pretrained model, covers the full temporal span via uniform sampling, and is the standard methodology across the video understanding research community.